BÀI TẬP THỰC HÀNH: XÂY DỰNG AI ĐỌC BIỂN SỐ NHÀ TỰ ĐỘNG (SVHN)

I. ĐỀ BÀI (TÌNH HUỐNG THỰC TẾ)

Bối cảnh: Bạn đang làm việc cho dự án bản đồ số của Google. Xe chụp ảnh đường phố (Street View Car) mang về hàng triệu bức ảnh các dãy nhà. Để định vị chính xác địa chỉ lên bản đồ, công ty cần một mô hình AI có thể tự động đọc các chữ số trên biển số nhà từ những bức ảnh này.

Nhiệm vụ của bạn: Xây dựng một mạng CNN để nhận diện chữ số từ bộ dữ liệu SVHN.

Thông tin Bộ dữ liệu (SVHN):

- Dữ liệu thực tế, ảnh màu RGB, kích thước 32×32 pixel.
- **Tập Train:** Hơn 73.000 ảnh. **Tập Test:** Hơn 26.000 ảnh.
- *Cạm bẫy dữ liệu (Data Trap):* Dữ liệu được lưu dưới dạng file MATLAB (.mat). Kích thước ma trận bị ngược so với chuẩn của Python, và nhãn (Label) cho số 0 lại được ký hiệu là số 10. Bạn phải tự tay xử lý các vấn đề này!

Mục tiêu KPI:

- Vượt qua ngưỡng độ chính xác Validation **85%**.

II. HƯỚNG DẪN THỰC HÀNH TỪNG BƯỚC (NOTEBOOK)

Bạn hãy chạy tuần tự các ô code dưới đây trên Google Colab.

[Text Cell 1: Tải dữ liệu thô từ máy chủ Stanford]

Bộ dữ liệu này không có sẵn lệnh load_data() trong Keras. Chúng ta sẽ dùng thư viện urllib để tải file .mat trực tiếp từ máy chủ của Đại học Stanford, và dùng scipy.io để đọc file.

[Code Cell 1]

Python
```
import urllib.request
import scipy.io as sio
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
```

```
# 1. Tải file dữ liệu .mat từ Stanford
train_url = 'http://ufldl.stanford.edu/housenumbers/train_32x32.mat'
test_url = 'http://ufldl.stanford.edu/housenumbers/test_32x32.mat'

print("Đang tải dữ liệu Train (khoảng 180MB)...")
urllib.request.urlretrieve(train_url, 'train_32x32.mat')
print("Đang tải dữ liệu Test (khoảng 64MB)...")
urllib.request.urlretrieve(test_url, 'test_32x32.mat')

# 2. Đọc file .mat
train_data = sio.loadmat('train_32x32.mat')
test_data = sio.loadmat('test_32x32.mat')

print("Đã tải và đọc file thành công!")
```

[Text Cell 2: Kỹ năng "Làm sạch" và Định hình dữ liệu (Data Wrangling)]

Khi in cấu trúc của train_data['X'], bạn sẽ thấy nó có hình dạng (32, 32, 3, 73257). Nghĩa là (Chiều cao, Chiều rộng, Kênh màu, Số lượng ảnh).

Tuy nhiên, mạng CNN của Keras bắt buộc định dạng phải là (Số lượng ảnh, Chiều cao, Chiều rộng, Kênh màu). Ta phải dùng lệnh np.transpose để xoay ma trận lại.

Đồng thời, trong SVHN gốc, chữ số 0 được gán nhãn là 10. Ta phải đổi toàn bộ số 10 thành 0 để phù hợp với hàm Softmax (từ 0-9).

[Code Cell 2]

Python

```
# Lấy mảng hình ảnh (X) và nhãn (y)
x_train = train_data['X']
y_train = train_data['y']
x_test = test_data['X']
y_test = test_data['y']

# 1. Định hình lại ma trận ảnh (Transpose)
# Đưa trục thứ 3 (Số lượng ảnh) lên vị trí đầu tiên (số 0)
x_train = np.transpose(x_train, (3, 0, 1, 2))
x_test = np.transpose(x_test, (3, 0, 1, 2))

# 2. Sửa lỗi nhãn số 0 (Đổi 10 thành 0)
y_train[y_train == 10] = 0
y_test[y_test == 10] = 0

# 3. Chuẩn hóa giá trị pixel về [0, 1]
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

print(f"Kích thước X_train sau khi sửa: {x_train.shape}") # (73257, 32, 32, 3)
print(f"Nhãn hiện tại gồm các số: {np.unique(y_train)}") # Sẽ hiển thị từ 0 đến 9
```

[Text Cell 3: Trực quan hóa dữ liệu thực tế]

Hãy xem thử những bức ảnh này "khắc nghiệt" đến mức nào nhé.

[Code Cell 3]

Python
```
plt.figure(figsize=(10, 4))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(x_train[i])
    plt.title(f"Nhãn: {y_train[i][0]}")
    plt.axis('off')
plt.tight_layout()
plt.show()
```
(Sinh viên sẽ nhận thấy ảnh bị nhòe, tối, và đôi khi nhãn là số 1 nhưng bên cạnh có cả số 2 và 3 lọt vào khung hình).

[Text Cell 4: Thiết kế Mạng CNN]

Với dữ liệu nhiễu như thế này, chúng ta cần một kiến trúc kết hợp cả **Batch Normalization** (để giữ cho việc học ổn định) và **Dropout** (để chống mô hình học vẹt các con số rác xung quanh).

[Code Cell 4]

Python
```
model = models.Sequential()

# Block 1
model.add(layers.Conv2D(32, (3, 3), padding='same', activation='relu', input_shape=(32, 32, 3)))
model.add(layers.BatchNormalization())
model.add(layers.Conv2D(32, (3, 3), padding='same', activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Dropout(0.25))

# Block 2
model.add(layers.Conv2D(64, (3, 3), padding='same', activation='relu'))
model.add(layers.BatchNormalization())
model.add(layers.Conv2D(64, (3, 3), padding='same', activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Dropout(0.25))

# Khối phân loại (Dense)
model.add(layers.Flatten())
model.add(layers.Dense(256, activation='relu'))
model.add(layers.BatchNormalization())
model.add(layers.Dropout(0.5))

# Lớp Output (10 số)
```

```
model.add(layers.Dense(10, activation='softmax'))

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

[Text Cell 5: Huấn luyện mô hình]

Bộ dữ liệu SVHN khá lớn (hơn 73k ảnh), nên chúng ta sẽ dùng batch_size lớn một chút (ví dụ 128 hoặc 256) để tăng tốc độ tính toán.

[Code Cell 5]

Python

```
from tensorflow.keras.callbacks import EarlyStopping

# Dừng sớm nếu sau 5 epochs không thấy cải thiện để tránh tốn thời gian
early_stopping = EarlyStopping(monitor='val_accuracy', patience=5, restore_best_weights=True)

print("Bắt đầu huấn luyện...")
history = model.fit(x_train, y_train,
                    epochs=20,
                    batch_size=128,
                    validation_split=0.1,
                    callbacks=[early_stopping])
print("Huấn luyện hoàn tất!")
```

[Text Cell 6: Đánh giá thành quả]

Chấm điểm trên hơn 26.000 ảnh Test hoàn toàn mới để xem AI của bạn có đạt KPI 85% không nhé!

[Code Cell 6]

Python

```
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f"🚀 Độ chính xác trên tập Test: {test_acc * 100:.2f}%")

# Vẽ biểu đồ Accuracy
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Độ chính xác qua các vòng lặp')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid()
plt.show()
```

III. THỬ THÁCH NÂNG CAO (LẤY ĐIỂM CỘNG)

Chủ đề: Màu sắc có thực sự quan trọng đối với chữ số?

SVHN là ảnh màu RGB (3 kênh). Tuy nhiên, để đọc một con số, con người thường chỉ quan tâm đến hình dáng (cạnh, góc) chứ không quan tâm chữ số đó màu đỏ hay xanh. Nếu giữ nguyên 3 kênh màu, mạng nơ-ron phải tính toán rất nhiều và có thể bị "rối" bởi màu sắc của biển số nhà.

- **Nhiệm vụ:** Trước khi đưa ảnh vào mô hình, hãy viết một đoạn code dùng numpy hoặc OpenCV để chuyển đổi toàn bộ x_train và x_test từ ảnh màu RGB thành ảnh xám Grayscale (1 kênh màu).
- Sau đó, thay đổi input_shape=(32, 32, 1), huấn luyện lại mô hình và quan sát: Tốc độ huấn luyện có nhanh hơn không? Độ chính xác có tăng lên (hoặc giữ nguyên) so với lúc dùng ảnh màu không?

Lưu ý:

1. **Bài học về Data Engineering:** Ở phần np.transpose, chúng ta thường rất lú lẫn về các con số (3, 0, 1, 2). Giải thích trực quan: Trục ban đầu là (H, W, Channels, Số ảnh). Chúng ta bốc trục số 3 (Số ảnh) đem lên đầu tiên, các trục 0, 1, 2 bị lùi ra sau, tạo thành (Số ảnh, H, W, Channels). Đây là lỗi rất hay gặp khi làm việc với các hệ thống dữ liệu cũ.
2. **Tại sao không cần Data Augmentation?** Trong bài này không sử dụng Data Augmentation, tại sao? Lý do là: Dữ liệu SVHN đã quá phong phú (hơn 73k ảnh) và bản thân dữ liệu gốc đã rất biến thiên (bị cắt lệch, méo), nên chỉ cần dùng Dropout và Batch Normalization là đủ sức đạt > 90% accuracy.